
Gradient Bitumen

Conversational Sales Agent

Solution Design & Plan

One natural-language prompt creates the full sales chain — Opportunity, Order, Loading Advice and its Line — and auto-submits it for approval. The agent asks for anything missing, confirms before it writes, creates everything in a single all-or-nothing transaction, and sends the user a confirmation message with links to the new records.

Org	salesgvgr — PartialSB sandbox (gb-partialsb), API 66.0
Front-end	Agentforce agent (utility bar + Account page); LWC chat fallback
Brains	Apex invocable action library; one transactional orchestration action
Status	Schema confirmed (Step 2 done); ready for Phase 1–2 build

1. How the agent behaves (the contract)

The agent is single-shot: the user describes the whole job in one message and the agent drives the entire chain. It only pauses to collect missing required fields and to get a go-ahead before writing.

1	User sends one prompt Natural language containing the customer and the deal details, e.g. product, quantity, price, load type. On the Account page the customer is taken from the record; from the utility bar the agent asks for the account name.
2	Agent parses & resolves Extracts values for all four records and resolves the Account (asking which one if the name is ambiguous).
3	Ask for any gaps — once Runs preview across all four records using the curated required-field list. Every missing required field is asked for together in a single follow-up, not one at a time. Loops until complete.
4	Confirm before writing Shows a single summary of everything it will create and waits for an explicit "yes". (Pre-create gate — see Open Items if you want it removed.)
5	Create — all or nothing Creates Opportunity → Order → Loading Advice → Line in one transaction with a savepoint; any failure rolls the whole thing back so no orphans are left. Sets the LA record type from the load type.
6	Auto-submit for approval Calls the headless approval path (internal PDF + notification + email; status set to Draft / Pending 1st Approver).
7	Confirmation message to the user Returns a success message listing the four created records with links, plus the submission status.

Worked example

User: "Create a commercial loading advice for A A Fugu & Sons – 30 MT of bitumen at NGN 450,000/MT, deliver to Kano."

Agent: "I can set that up. A couple of things first:
 - What close date should I use for the Opportunity?
 - What Order start date?
 - Customer Type for the Loading Advice?"

User: "Close 30/06, order starts today, customer type Reseller."

Agent: "Here is what I will create for A A Fugu & Sons Ltd:
 Opportunity New Business, NGN 13,500,000, close 30/06/2026
 Order starts 01/06/2026, status Draft
 Loading Advice Commercial - 30 MT Bitumen @ 450,000, Kano
 Then auto-submit for approval. Proceed? (yes / no)"

User: "yes"

Agent: "Done. Created Opportunity OPP-1042, Order 00000356, Loading Advice LA-0231 (+ line). Submitted to Joy for approval - PDF, bell alert and email sent."

2. Confirmed ground truth (from Step 2)

These facts were verified against the org via `sf subject describe` and retrieved metadata, and are pinned in `CLAUDE.md` so every session uses them.

Fact	Confirmed value
Opportunity → Account	Lookup (AccountId)
Order → Opportunity / Account	Lookups; AccountId effectively mandatory at insert — agent always sets it
Loading_Advice__c → Order	Master-Detail via Order_DKL__c (non-reparentable, cascade)
Loading_Advice_Line__c → Loading_Advice__c	Master-Detail via LoadingAdvice_DKL__c
Order mandatory at creation	EffectiveDate (start date) + Status (and AccountId in practice)
Load type → Record Type	Commercial → Commercial_DKL; Stock Transfer → StockTransfer_DKL
Dispatch PDF routing	By record type, fires later at approval (LA_Approve_Reject) — NOT part of submit
Header guard	Lock_when_Approved validation rule (relevant only to future update actions)

Required fields = business rule, not the DB

At database level only `Customer_DKL__c` and the master-detail link are non-nullable on the Line. The agent must instead enforce the **curated** required set the Create-LA screen uses: **Customer, Customer Type, Location, Product, Quantity (MT), Price (N/MT)**. This list is the single source of truth for what counts as “missing”.

3. Architecture

- **Conversation layer (Agentforce):** NLU, the utility-bar & Account-page surfaces, conversational memory, and insight answers. Configured with the actions below as tools and instructions that enforce the §1 contract.
- **Action layer (Apex invocable):** deterministic, unit-tested methods that validate and write. The chain logic lives here — not in the prompt — so it is testable and certifiable.
- **Fallback front-end:** a custom LWC chat panel + Apex LLM controller (Named Credential) using function-calling to the same actions, if Agentforce is not licensed.

Action catalogue (each ships with a test class)

Action	Role
Agent_ResolveAccount	Resolve customer by recordId or name; disambiguate
Agent_GetAccountInsight	Read-only insight: open opps, order status, recent loading advices, terms
Agent_DescribeObject	Explain an object and its required fields
Agent_PreviewSalesChain	Validate all four records from the parsed prompt; return ONE consolidated missingFields list + a single summary
Agent_CommitSalesChain	With confirmation token: create Opp→Order→LA→Line in one transaction (savepoint + rollback), set LA record type, then call submit; return record links
Agent_SubmitLAForApproval	Run the headless approval flow (PDF + notification + email + status)

Shared result wrapper

```
success(Boolean) | status(OK|NEEDS_INPUT|NEEDS_CONFIRM|ERROR) | recordId | recordUrl
missingFields(List<String>) | summaryJson | message
```

4. Transaction & error handling

Because the relationships are master-detail, the children need their parent Id at insert, and a half-built chain would leave orphaned records. The commit is therefore atomic.

- 1 Database.setSavepoint() at the start of **Agent_CommitSalesChain**.
- 2 Insert Opportunity → capture Id → insert Order (with Opp + Account) → capture Id → insert Loading Advice (with Order_DKL__c + RecordTypeId) → capture Id → insert Line (with LoadingAdvice_DKL__c).
- 3 Any DML error → Database.rollback(sp), create nothing, return status **ERROR** with the cause.
- 4 On success → call **Agent_SubmitLAForApproval**, then return all four record links and the submit status.
- 5 Commit refuses to run without a valid confirmation token (tested invariant).

Test coverage required before each deploy

Happy path (4 created + submitted) · each missing-field branch (consolidated ask, nothing created) · commit-without-confirmation rejection · mid-chain failure rolls everything back cleanly. Target ≥90% on new code to keep the org above the 75% gate.

5. Build phases

Phase 1 — Read-only foundation

- 1 Agent_ResolveAccount, Agent_GetAccountInsight, Agent_DescribeObject + tests; deploy & demo.

Phase 2 — Single-prompt chain

- 1 Per-object preview validators using the curated required list.
- 2 Agent_PreviewSalesChain (consolidated) and Agent_CommitSalesChain (transactional, sets record type).
- 3 Full test matrix from §4.

Phase 3 — Headless auto-approval

- 1 Autolaunched flow mirroring ONLY the Submit-for-Approval screen flow; wrap in Agent_SubmitLAForApproval; verify PDF + bell + email.

Phase 4 — Conversation layer

- 1 Build/wire the Agentforce agent (or LWC fallback); instructions enforce ask-missing → confirm → commit → confirmation message; place on utility bar + Account page.

Phase 5 — Hardening & handover

- 1 End-to-end test from both surfaces; ≥75% org coverage; FLS/sharing review; handover doc + any Agent Builder/licensing notes.

6. Open items & assumptions

Item	Default taken	Need from you
Pre-create confirmation gate	Kept (your original brief said 'confirm before creating')	Keep the yes/no gate, or create immediately once fields are complete?
Existing vs fresh Opp/Order	Always create fresh each run	Should the agent reuse an existing open Opportunity/Order if one matches?
Confirmation message channel	In-chat message + record links	Also email the user, or chat only?
Agentforce licensing	Assumed available	Confirm; otherwise we use the LWC fallback

Item	Default taken	Need from you
Update/delete via agent	Out of v1	Confirm deferred

The matching Claude Code prompt for Phases 1–2 is provided as a separate file (Claude_Code_Phase2_Prompt.txt) and reflects this design exactly.